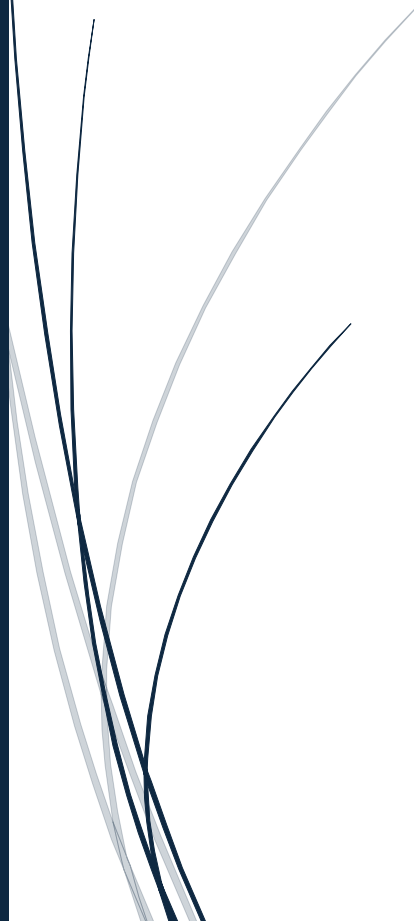




Digital Calculator using FPGA



by
SRUTHI B
RANJANA S

DIGITAL CALCULATOR USING INTEL QUARTUS & DE10 LITE FPGA

S.NO	CONTENT	PAGE NO
1.	Verilog Code for digital calculator	4
2.	RTL viewer	9
3.	Pin planner	10
4.	Analysis & synthesis	12
5.	Tech map viewer – post map	13
6.	Fitter	15
7.	Chip planner	16
8.	Assembler	18
9.	Output	19
10.	Conclusion	24

ABSTRACT

The project is to design and implement a Digital Calculator using Intel max 10 FPGA and Intel quartus software. This project presents the design, development, and implementation of a fully functional digital calculator system using Intel MAX 10 FPGA (10M50DAF484C7G) on the DE10-Lite development board. The calculator performs four fundamental arithmetic operations: addition, subtraction, multiplication, and division, with 8-bit input capability and 16-bit result display.

The system utilizes the limited I/O resources available on the DE10-Lite board, employing 10 toggle switches for data input and operation selection, and 2 push buttons for operand latching and mode selection. The calculator features an intelligent input mechanism where switches SW0-SW7 serve as data input, while SW8-SW9 function as operation selectors. The two push buttons implement a smart latching system for storing two distinct operands and toggling between input modes.

The design is implemented in Verilog HDL and demonstrates efficient digital logic design principles, including sequential logic for input latching, combinatorial logic for arithmetic operations, and display drivers for seven-segment output. The calculator displays results across six seven-segment displays, showing both input operands and computed results in real-time.

Key achievements include successful implementation of all four arithmetic operations with correct 16-bit result handling for multiplication, efficient resource utilization of the FPGA, and a user-friendly interface despite limited input peripherals. The project showcases practical application of digital system design concepts, FPGA programming, and hardware-software co-design using Intel Quartus Prime software tools.

The implemented calculator system proves the capability of FPGAs in creating custom digital computing systems and serves as a foundation for more complex arithmetic processors. The project successfully demonstrates the integration of multiple digital design components including state machines, arithmetic units, and display controllers within a single FPGA fabric.

INTRODUCTION

Project Overview

This project involves the design and implementation of a fully functional digital calculator system using Field Programmable Gate Array (FPGA) technology. The calculator is developed for the Intel MAX 10 FPGA (10M50DAF484C7G) mounted on the DE10-Lite development board, which serves as the hardware platform for this digital system. The implemented calculator performs four fundamental arithmetic operations—addition, subtraction, multiplication, and division—on 8-bit binary numbers and displays the results in real-time on six seven-segment displays.

The primary objective of this project is to demonstrate practical application of digital logic design principles, hardware description languages, and FPGA programming to create a custom computing system. The calculator represents a complete digital system that integrates multiple components including input interfaces, arithmetic logic units, control units, and output display drivers within a single FPGA fabric.

Hardware Configuration

- Platform: Intel MAX 10 FPGA (10M50DAF484C7G)
- Programming: Verilog Hardware Description Language
- Clock: 50Hz system frequency

Switches Configuration:

- 8 data switches (SW0-SW7) for number input (0-255 range)
- 2 operation switches (SW8-SW9) for function selection:
 - 00 = Addition
 - 01 = Subtraction
 - 10 = Multiplication
 - 11 = Division

Push Buttons:

- KEY0: Store/Enter button - latches current number
- KEY1: Toggle button - switches between Operand A and B input

Output Display

Six Seven-Segment Displays:

- HEX5: Shows input mode (A = first number, b = second number)
- HEX4: Displays operation code (0,1,2,3)
- HEX3-HEX0: Shows 16-bit result in hexadecimal format

1) Verilog code for Digital Calculator

Main Calculator module:

Module Interface:

The main calculator module serves as the core processing unit with the following I/O configuration:

- Clock Input: 50MHz system clock for synchronous operation
- Switch Inputs: 10-bit input where SW[7:0] are data bits and SW[9:8] are operation select bits
- Button Inputs: Two push buttons (KEY[1:0]) for operand storage control
- Display Outputs: Six 7-segment display drivers (HEX0-HEX5)

Internal Register Architecture:

The module employs four key registers:

- OperandA & OperandB: 8-bit registers storing user-input numbers
- Result: 16-bit register holding arithmetic computation results
- Display_value: 16-bit register for output formatting

Arithmetic Logic Unit (ALU) Design

The ALU implements a combinational logic block with four operations:

Operation Encoding:

- 2'b00: Addition with zero-extension
- 2'b01: Subtraction with zero-extension
- 2'b10: 8×8 bit multiplication
- 2'b11: Division with zero-operand protection

Display Management System:

- HEX0-HEX1: Lower byte of result (bits [7:0])
- HEX2-HEX3: Higher byte of result (bits [15:8])
- HEX4-HEX5: Operand B display (for user reference)

Verilog Code:

```
module calculator(  
    input clk,  
    input [9:0] SW,  
    input [1:0] KEY, // KEY[0] and KEY[1] for latching numbers  
    output [6:0] HEX0, HEX1, HEX2, HEX3, HEX4, HEX5  
);  
  
    reg [7:0] operandA, operandB;
```

```

wire [1:0] operation = SW[9:8];

reg [15:0] result;
reg [15:0] display_value; // Changed to 16-bit for multiplication

// Latch numbers using push buttons
always @(posedge clk) begin
    if (~KEY[0]) begin // KEY[0] pressed - latch first number
        operandA <= SW[7:0];
    end
    if (~KEY[1]) begin // KEY[1] pressed - latch second number
        operandB <= SW[7:0];
    end
end

// Perform operations
always @(*) begin
    case(operation)
        2'b00: result = {8'b0, operandA} + {8'b0, operandB}; // A + B
        2'b01: result = {8'b0, operandA} - {8'b0, operandB}; // A - B
        2'b10: result = operandA * operandB; // A * B
        2'b11: result = (operandB != 0) ? {8'b0, (operandA / operandB)} : 16'b0; // A / B
    endcase
end

// Display the full 16-bit result for all operations
always @(*) begin
    display_value = result;
end

// Display configuration:
// HEX0-HEX1: Result LOW byte
// HEX2-HEX3: Result HIGH byte (for multiplication)
// HEX4: Operand B (latched)
// HEX5: Operand A (latched)

seg7_display seg0(.in(display_value[3:0]), .out(HEX0));
seg7_display seg1(.in(display_value[7:4]), .out(HEX1));
seg7_display seg2(.in(display_value[11:8]), .out(HEX2));
seg7_display seg3(.in(display_value[15:12]), .out(HEX3));
seg7_display seg4(.in(operandB[3:0]), .out(HEX4));
seg7_display seg5(.in(operandB[7:4]), .out(HEX5));
endmodule

```

Segment display:

Decoder Architecture

The module implements a hexadecimal to 7-segment decoder using combinational logic:

Segment Encoding:

- Active-low configuration (0 = segment ON, 1 = segment OFF)
- Binary to hexadecimal conversion (0-F)
- Default case for blank display

Pattern Examples:

- '0': 7'b1000000 (segments a,b,c,d,e,f ON)
- '1': 7'b1111001 (segments b,c ON)
- 'A': 7'b0001000 (segments a,b,c,e,f,g ON)

Verilog Code:

```
module seg7_display(  
    input [3:0] in,  
    output reg [6:0] out  
);  
  
    always @(*) begin  
        case(in)  
            4'h0: out = 7'b1000000; // 0  
            4'h1: out = 7'b1111001; // 1  
            4'h2: out = 7'b0100100; // 2  
            4'h3: out = 7'b0110000; // 3  
            4'h4: out = 7'b0011001; // 4  
            4'h5: out = 7'b0010010; // 5  
            4'h6: out = 7'b0000010; // 6  
            4'h7: out = 7'b1111000; // 7  
            4'h8: out = 7'b0000000; // 8  
            4'h9: out = 7'b0010000; // 9  
            4'hA: out = 7'b0001000; // A  
            4'hB: out = 7'b0000011; // B  
            4'hC: out = 7'b1000110; // C  
            4'hD: out = 7'b0100001; // D  
            4'hE: out = 7'b0000110; // E  
            4'hF: out = 7'b0001110; // F  
            default: out = 7'b1111111; // Blank  
        endcase  
    end  
endmodule
```

Toplevel:

System Integration

The top module serves as the hardware interface layer:

Port Mapping:

- Direct connection to physical board resources
- Clock signal routing to calculator core
- Switch and button interface management
- Display output distribution

Design Benefits:

- Modular architecture for easy maintenance
- Clear separation between core logic and hardware interface
- Scalable design for future enhancements

Verilog Code:

```
module calc (  
    input MAX10_CLK1_50,  
    input [9:0] SW,  
    input [1:0] KEY, // Push buttons  
    output [6:0] HEX0, HEX1, HEX2, HEX3, HEX4, HEX5  
);  
  
    calculator calc_inst(  
        .clk(MAX10_CLK1_50),  
        .SW(SW),  
        .KEY(KEY),  
        .HEX0(HEX0),  
        .HEX1(HEX1),  
        .HEX2(HEX2),  
        .HEX3(HEX3),  
        .HEX4(HEX4),  
        .HEX5(HEX5)  
    );  
  
endmodule
```

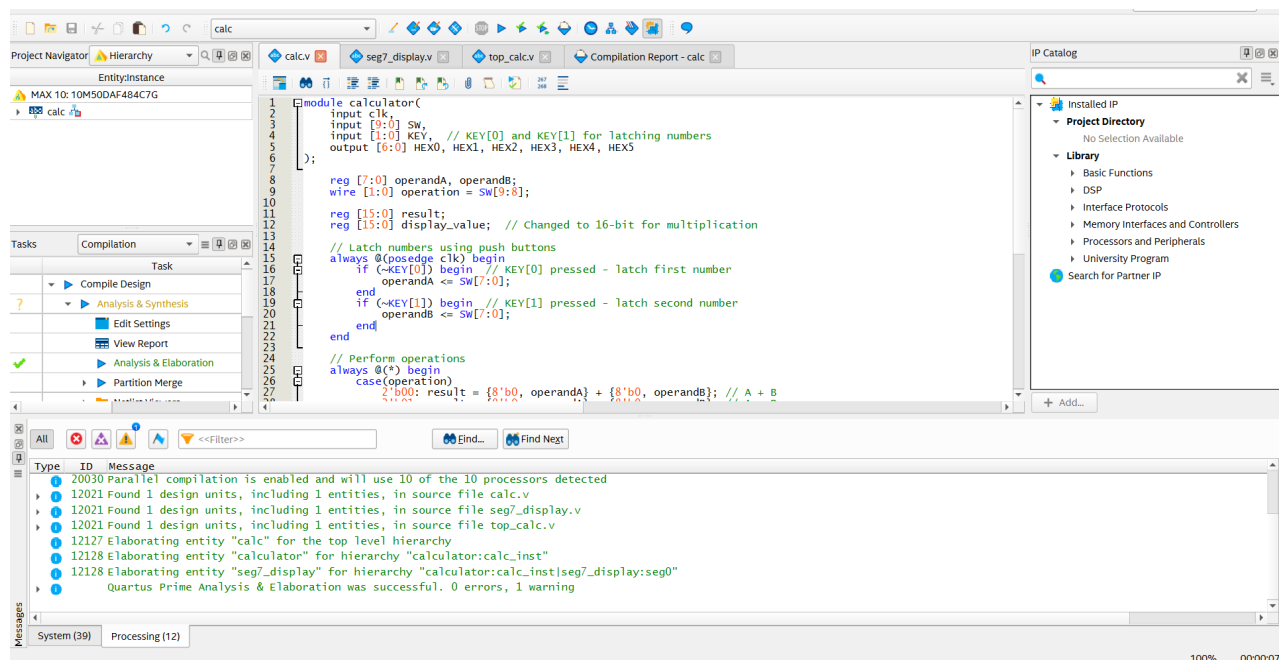


Figure 1.1: Verilog code

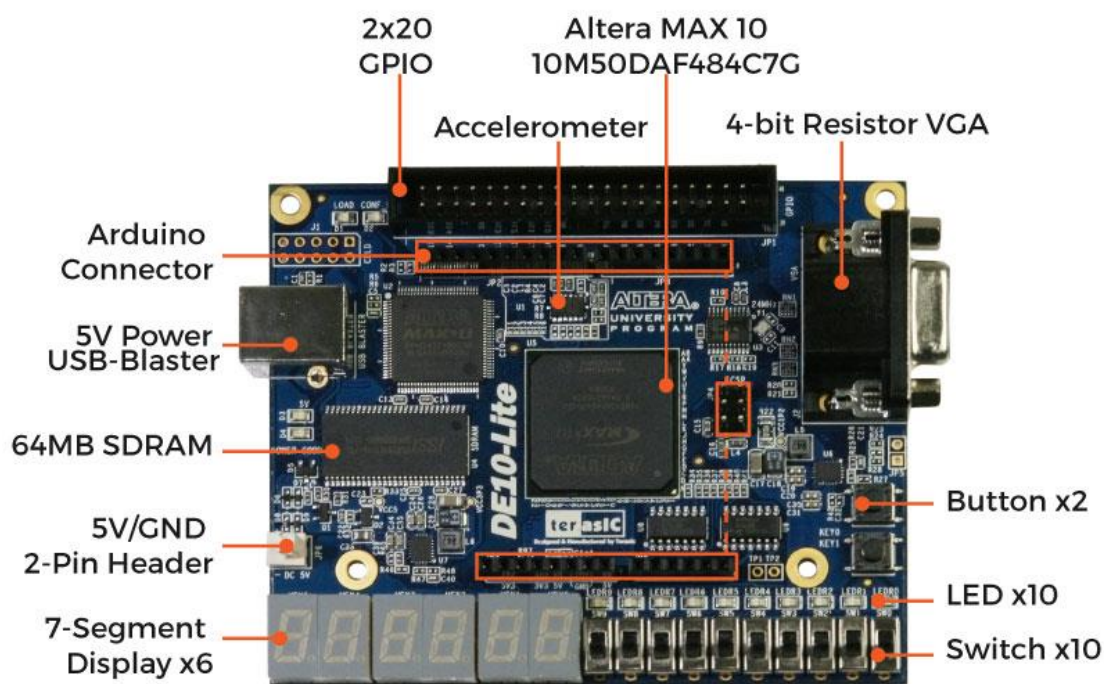


Figure 1.2: DE 10 lite -10M50DAF484C7G fpga

2) RTL viewer:

The RTL (Register Transfer Level) Viewer in Intel Quartus Prime is an essential tool for visualizing the gate-level schematic of synthesized digital designs. It provides a graphical representation of how the Verilog code is translated into actual digital logic components, offering valuable insights into the design's hardware implementation.

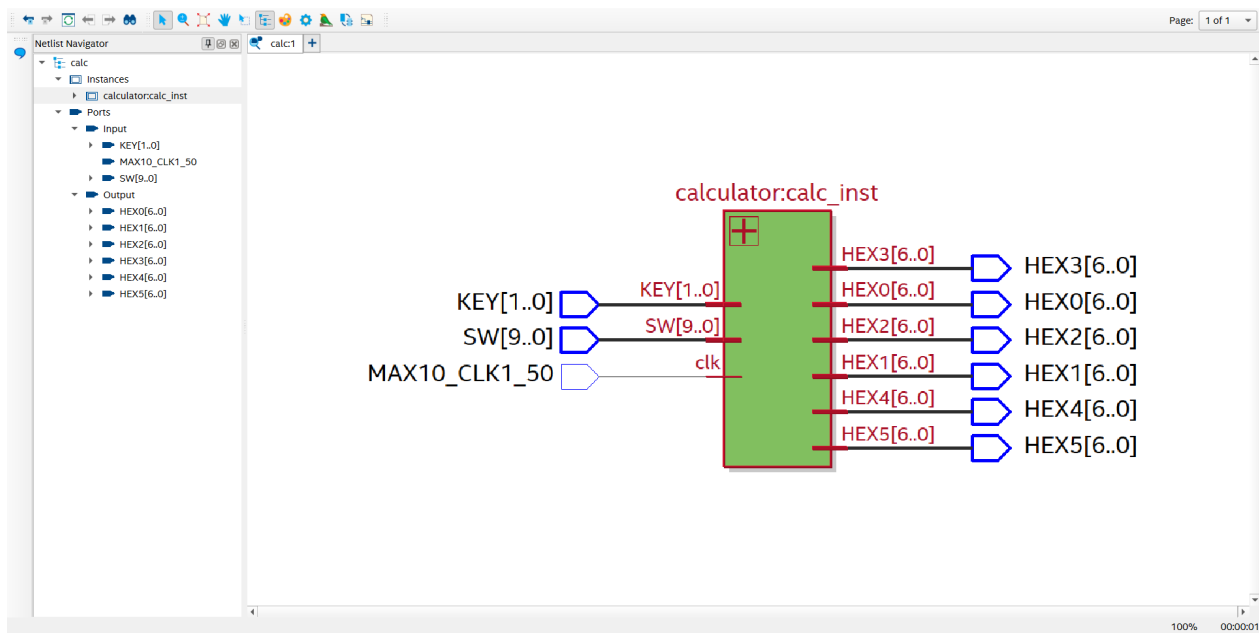


Figure 2.1: RTL viewer

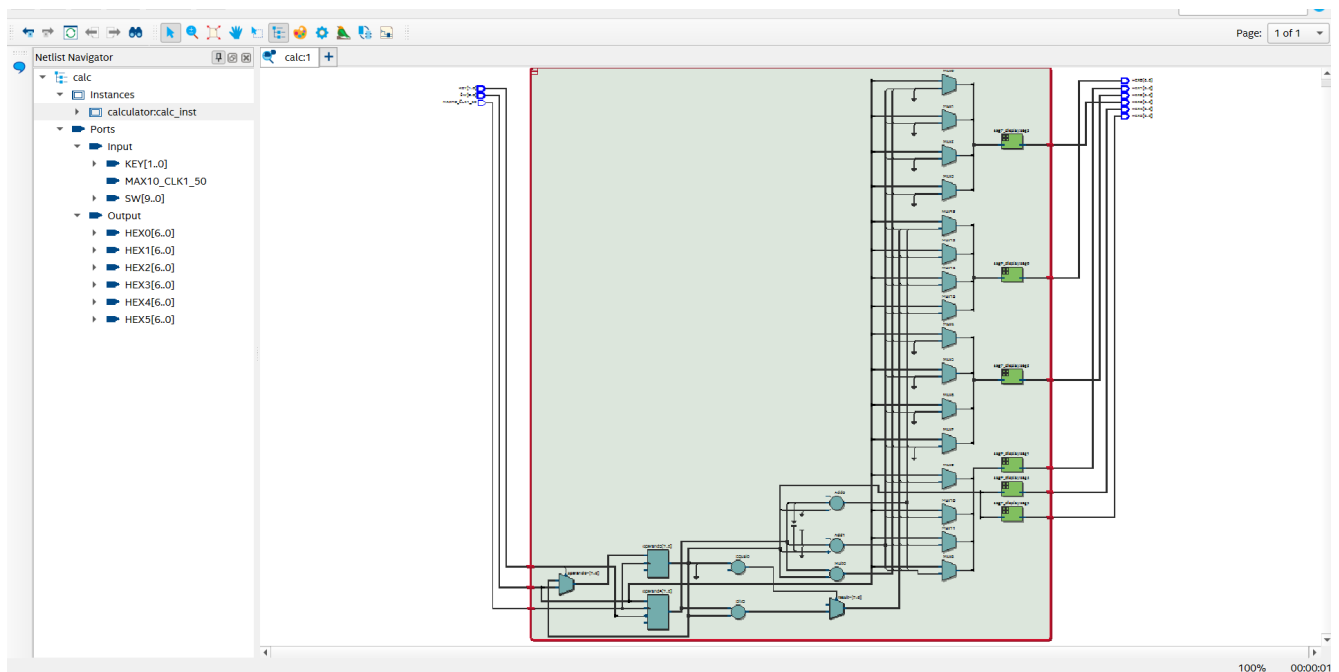


Figure 2.2: Gate level schematic

The Pin Planner in Intel Quartus Prime is an essential graphical interface tool used to assign physical FPGA pins to the logical ports defined in the Verilog design. It serves as the crucial bridge between the abstract digital design and the actual hardware implementation on the DE10-Lite board.

- **Signal Integrity:** Ensuring clean signal transmission
- **Timing Constraints:** Meeting setup and hold time requirements
- **Board Compatibility:** Matching FPGA pins with board layout
- **Power Management:** Proper distribution of power and ground connections

The pin assignment followed these systematic steps:

1. Design Analysis: Reviewed all input and output ports from Verilog modules
2. Board Schematic Reference: Consulted DE10-Lite board documentation
3. Pin Mapping: Matched logical ports to physical FPGA pins
4. Constraint Validation: Verified assignments against board capabilities

10

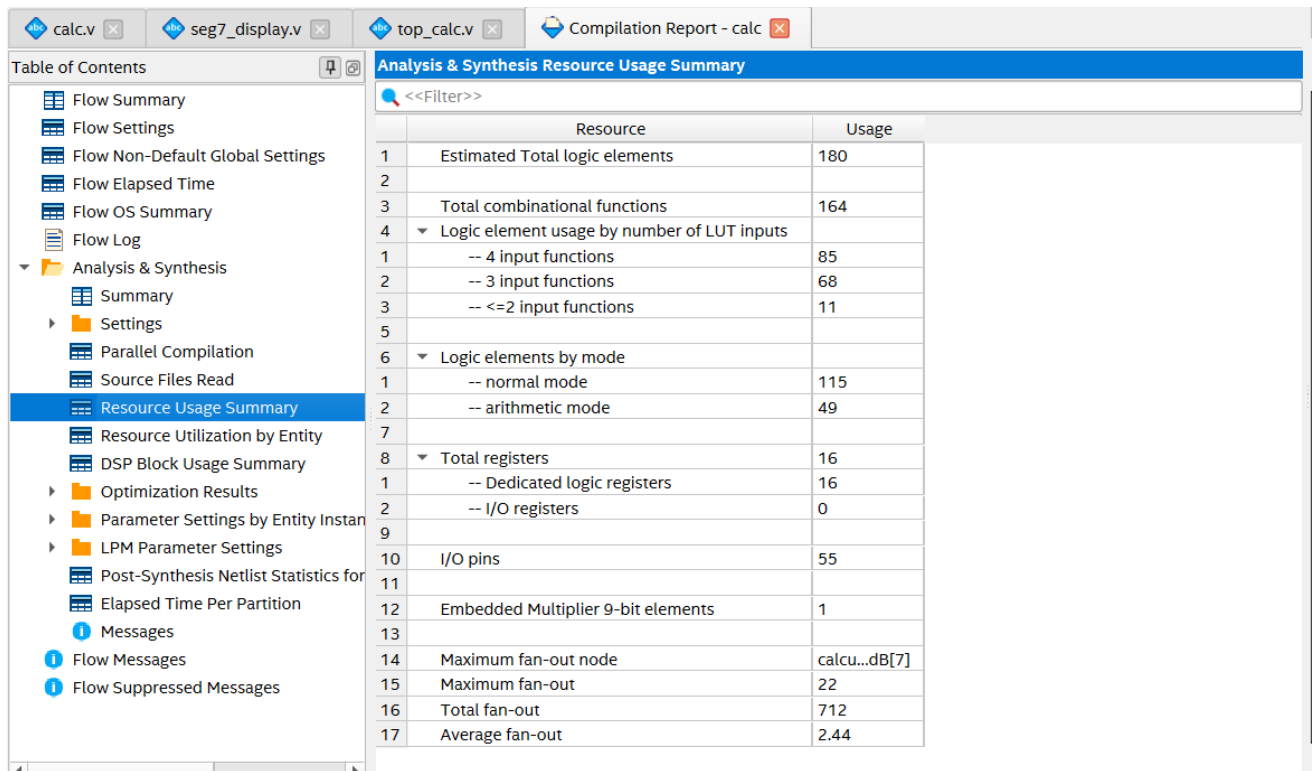
Named: * Edit: 											
Node Name	Direction	Location	I/O Bank	VREF Group	I/O Standard	Reserved	Current Strength	Slew Rate	Differential Pair	Strict Preservation	
OUT HEX0[6]	Output	PIN_C17	7	B7_NO	3.3-V LVTTTL		8mA (default)	2 (default)			
OUT HEX0[5]	Output	PIN_D17	7	B7_NO	3.3-V LVTTTL		8mA (default)	2 (default)			
OUT HEX0[4]	Output	PIN_E16	7	B7_NO	3.3-V LVTTTL		8mA (default)	2 (default)			
OUT HEX0[3]	Output	PIN_C16	7	B7_NO	3.3-V LVTTTL		8mA (default)	2 (default)			
OUT HEX0[2]	Output	PIN_C15	7	B7_NO	3.3-V LVTTTL		8mA (default)	2 (default)			
OUT HEX0[1]	Output	PIN_E15	7	B7_NO	3.3-V LVTTTL		8mA (default)	2 (default)			
OUT HEX0[0]	Output	PIN_C14	7	B7_NO	3.3-V LVTTTL		8mA (default)	2 (default)			
OUT HEX1[6]	Output	PIN_B17	7	B7_NO	3.3-V LVTTTL		8mA (default)	2 (default)			
OUT HEX1[5]	Output	PIN_A18	7	B7_NO	3.3-V LVTTTL		8mA (default)	2 (default)			
OUT HEX1[4]	Output	PIN_A17	7	B7_NO	3.3-V LVTTTL		8mA (default)	2 (default)			
OUT HEX1[3]	Output	PIN_B16	7	B7_NO	3.3-V LVTTTL		8mA (default)	2 (default)			
OUT HEX1[2]	Output	PIN_E18	6	B6_NO	3.3-V LVTTTL		8mA (default)	2 (default)			
OUT HEX1[1]	Output	PIN_D18	6	B6_NO	3.3-V LVTTTL		8mA (default)	2 (default)			
OUT HEX1[0]	Output	PIN_C18	7	B7_NO	3.3-V LVTTTL		8mA (default)	2 (default)			
OUT HEX2[6]	Output	PIN_B22	6	B6_NO	3.3-V LVTTTL		8mA (default)	2 (default)			
OUT HEX2[5]	Output	PIN_C22	6	B6_NO	3.3-V LVTTTL		8mA (default)	2 (default)			
OUT HEX2[4]	Output	PIN_B21	6	B6_NO	3.3-V LVTTTL		8mA (default)	2 (default)			
OUT HEX2[3]	Output	PIN_A21	6	B6_NO	3.3-V LVTTTL		8mA (default)	2 (default)			
OUT HEX2[2]	Output	PIN_B19	7	B7_NO	3.3-V LVTTTL		8mA (default)	2 (default)			
OUT HEX2[1]	Output	PIN_A20	7	B7_NO	3.3-V LVTTTL		8mA (default)	2 (default)			
OUT HEX2[0]	Output	PIN_B20	6	B6_NO	3.3-V LVTTTL		8mA (default)	2 (default)			
OUT HEX3[6]	Output	PIN_E17	6	B6_NO	3.3-V LVTTTL		8mA (default)	2 (default)			
OUT HEX3[5]	Output	PIN_D19	6	B6_NO	3.3-V LVTTTL		8mA (default)	2 (default)			
OUT HEX3[4]	Output	PIN_C20	6	B6_NO	3.3-V LVTTTL		8mA (default)	2 (default)			
OUT HEX3[3]	Output	PIN_C19	7	B7_NO	3.3-V LVTTTL		8mA (default)	2 (default)			
OUT HEX3[2]	Output	PIN_E21	6	B6_NO	3.3-V LVTTTL		8mA (default)	2 (default)			
OUT HEX3[1]	Output	PIN_E22	6	B6_NO	3.3-V LVTTTL		8mA (default)	2 (default)			
OUT HEX3[0]	Output	PIN_F21	6	B6_NO	3.3-V LVTTTL		8mA (default)	2 (default)			
OUT HEX4[6]	Output	PIN_F20	6	B6_NO	3.3-V LVTTTL		8mA (default)	2 (default)			
OUT HEX4[5]	Output	PIN_F19	6	B6_NO	3.3-V LVTTTL		8mA (default)	2 (default)			
OUT HEX4[4]	Output	PIN_H19	6	B6_NO	3.3-V LVTTTL		8mA (default)	2 (default)			
OUT HEX4[3]	Output	PIN_J18	6	B6_NO	3.3-V LVTTTL		8mA (default)	2 (default)			
OUT HEX4[2]	Output	PIN_E19	6	B6_NO	3.3-V LVTTTL		8mA (default)	2 (default)			
OUT HEX4[1]	Output	PIN_E20	6	B6_NO	3.3-V LVTTTL		8mA (default)	2 (default)			
OUT HEX4[0]	Output	PIN_F18	6	B6_NO	3.3-V LVTTTL		8mA (default)	2 (default)			
OUT HEX5[6]	Output	PIN_N20	6	B6_NO	3.3-V LVTTTL		8mA (default)	2 (default)			
OUT HEX5[5]	Output	PIN_N19	6	B6_NO	3.3-V LVTTTL		8mA (default)	2 (default)			
OUT HEX5[4]	Output	PIN_M20	6	B6_NO	3.3-V LVTTTL		8mA (default)	2 (default)			
OUT HEX5[3]	Output	PIN_N18	6	B6_NO	3.3-V LVTTTL		8mA (default)	2 (default)			
OUT HEX5[2]	Output	PIN_L18	6	B6_NO	3.3-V LVTTTL		8mA (default)	2 (default)			
OUT HEX5[1]	Output	PIN_K20	6	B6_NO	3.3-V LVTTTL		8mA (default)	2 (default)			
OUT HEX5[0]	Output	PIN_J20	6	B6_NO	3.3-V LVTTTL		8mA (default)	2 (default)			
IN KEY[1]	Input	PIN_A7	7	B7_NO	3.3 V Sc... Trigger		8mA (default)				
IN KEY[0]	Input	PIN_B8	7	B7_NO	3.3 V Sc... Trigger		8mA (default)				
IN MAX10_CLK1_50	Input	PIN_A11	7	B7_NO	3.3-V LVTTTL		8mA (default)				
IN SW[9]	Input	PIN_F15	7	B7_NO	3.3-V LVTTTL		8mA (default)				
IN SW[8]	Input	PIN_B14	7	B7_NO	3.3-V LVTTTL		8mA (default)				
IN SW[7]	Input	PIN_A14	7	B7_NO	3.3-V LVTTTL		8mA (default)				
IN SW[6]	Input	PIN_A13	7	B7_NO	3.3-V LVTTTL		8mA (default)				
IN SW[5]	Input	PIN_B12	7	B7_NO	3.3-V LVTTTL		8mA (default)				
IN SW[4]	Input	PIN_A12	7	B7_NO	3.3-V LVTTTL		8mA (default)				
IN SW[3]	Input	PIN_C12	7	B7_NO	3.3-V LVTTTL		8mA (default)				
IN SW[2]	Input	PIN_D12	7	B7_NO	3.3-V LVTTTL		8mA (default)				
IN SW[1]	Input	PIN_C11	7	B7_NO	3.3-V LVTTTL		8mA (default)				
IN SW[0]	Input	PIN_C10	7	B7_NO	3.3-V LVTTTL		8mA (default)				

Figure 3.2: Pin planner

4)Analysis & synthesis:

Analysis & Synthesis is the first major stage in the compilation flow where the Verilog HDL code is converted into a technology-mapped netlist optimized for the target MAX 10 FPGA architecture.

After Analysis and Synthesis process, in flow summary we can see Resource usage details. Resource usage detail shows how many logic elements used, total combinational function, total registers, I/O ports and fan out node.



	Resource	Usage
1	Estimated Total logic elements	180
2		
3	Total combinational functions	164
4	▼ Logic element usage by number of LUT inputs	
1	-- 4 input functions	85
2	-- 3 input functions	68
3	-- <=2 input functions	11
5		
6	▼ Logic elements by mode	
1	-- normal mode	115
2	-- arithmetic mode	49
7		
8	▼ Total registers	16
1	-- Dedicated logic registers	16
2	-- I/O registers	0
9		
10	I/O pins	55
11		
12	Embedded Multiplier 9-bit elements	1
13		
14	Maximum fan-out node	calcu...dB[7]
15	Maximum fan-out	22
16	Total fan-out	712
17	Average fan-out	2.44

Figure 4.1: Resource usage details in Analysis and synthesis

5)Tech map viewer (post map):

The Technology Map Viewer in Intel Quartus Prime displays the final implementation of the design after technology mapping, showing how the logical design has been mapped onto the actual FPGA hardware resources. This post-mapping view reveals the specific primitive elements and their interconnections within the target MAX 10 FPGA architecture.

- Converts your abstract code (like +, -, *) into actual electronic circuits
- Displays the physical logic gates, registers, and interconnections that were created from your Verilog
- Shows exactly how the FPGA's internal resources (LUTs, flip-flops, carry chains) are being used

Before Mapping:

- You see logical blocks (adders, multipliers, registers)
- Abstract representation of your design

After Mapping:

- You see real FPGA components:
 - LUTs (Look-Up Tables) that implement your logic
 - Flip-flops that store your data
 - Carry chains for arithmetic operations
 - I/O buffers for input/output signals

Arithmetic Units:

- How the 8-bit adder is built using carry chain logic
- How the multiplier uses multiple LUTs in parallel
- How the division logic is implemented with comparators and multiplexers

Display System:

- How each 7-segment decoder uses 7 separate LUTs
- How the output signals are routed to physical pins

Control Logic:

- How the button presses are synchronized with clock signals
- How the operation selection uses multiplexer trees

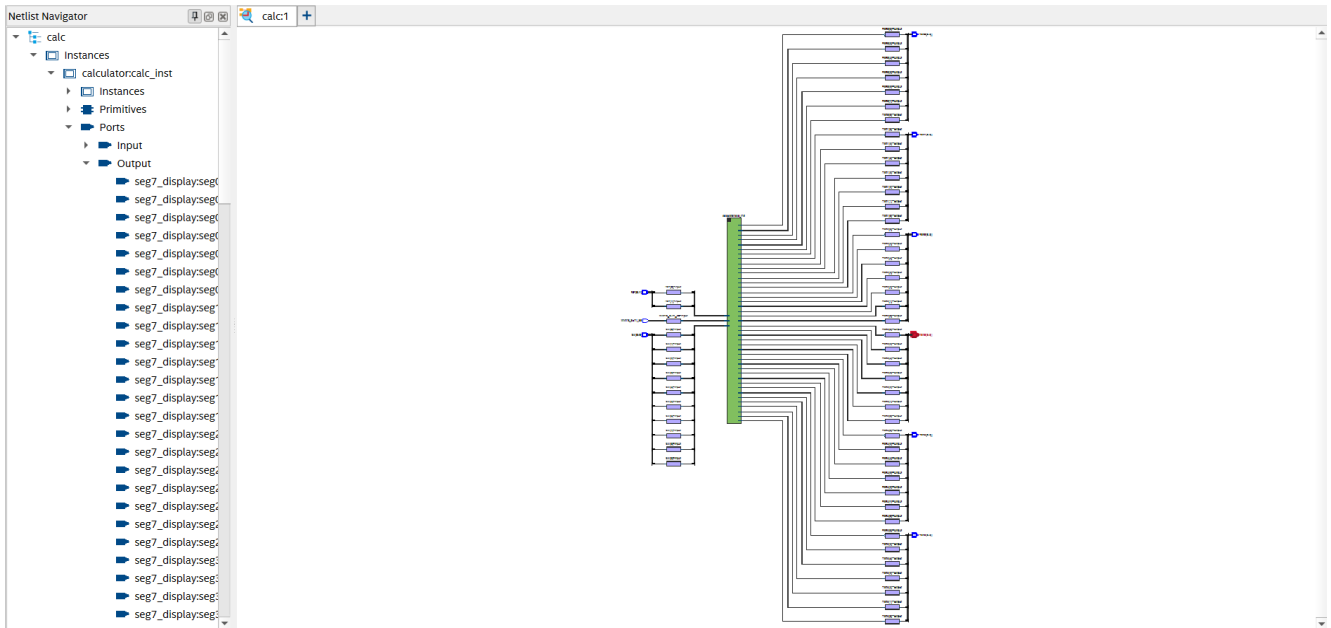


Figure 5.1: Tech map viewer post map

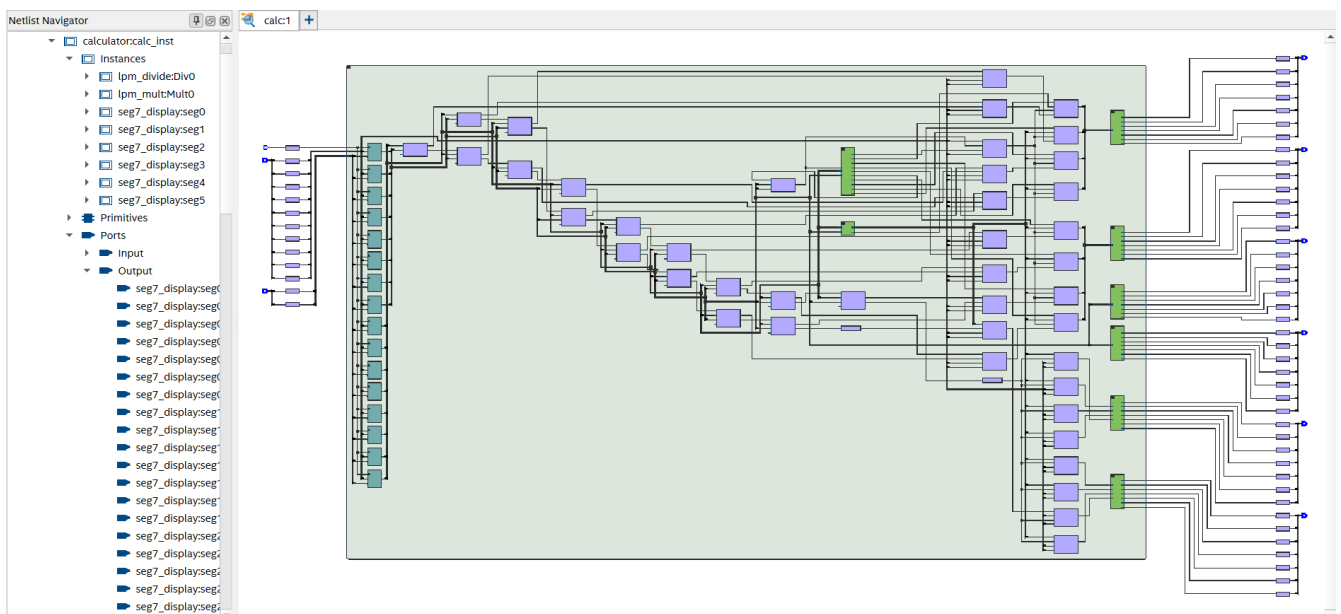


Figure 5.2 Internal structure of Calculator in tech map view

6)Fitter:

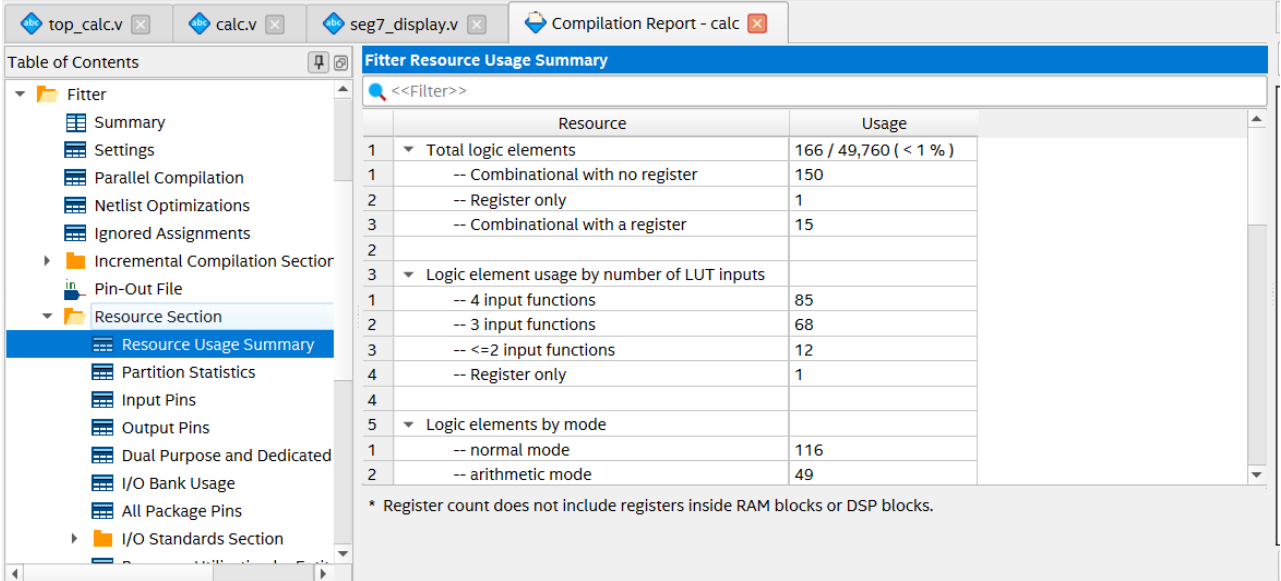
The Fitter in Intel Quartus Prime is the placement and routing engine that takes the synthesized netlist and maps it onto the physical resources of the target FPGA device. It determines the optimal placement of logic elements and establishes the routing connections between them to meet timing constraints and design requirements.

Fitting Process Stages:

1. Placement Phase: Logic elements assigned to specific LABs and ALMs
2. Routing Phase: Interconnections established using routing resources
3. Timing Analysis: Verification of timing constraints
4. Optimization: Iterative improvements to meet design goals

The Fitter operates after Analysis & Synthesis and is responsible for:

- Placement: Assigning logic elements to specific locations on the FPGA
- Routing: Creating physical connections between placed elements
- Timing Optimization: Ensuring design meets timing constraints
- Resource Allocation: Efficiently utilizing FPGA resources



Resource		Usage
1	▼ Total logic elements	166 / 49,760 (< 1 %)
1	-- Combinational with no register	150
2	-- Register only	1
3	-- Combinational with a register	15
2		
3	▼ Logic element usage by number of LUT inputs	
1	-- 4 input functions	85
2	-- 3 input functions	68
3	-- <=2 input functions	12
4	-- Register only	1
4		
5	▼ Logic elements by mode	
1	-- normal mode	116
2	-- arithmetic mode	49

* Register count does not include registers inside RAM blocks or DSP blocks.

Figure 6.1: Fitter – Resource usage summary

7)Chip Planner:

The Chip Planner is a graphical analysis tool in Intel Quartus Prime that provides a physical view of the FPGA device, showing exactly how your design has been placed and routed on the actual silicon. It displays the detailed layout of logic elements, memory blocks, DSP blocks, and routing resources.

The Chip Planner shows our calculator design minimally utilizes the MAX 10 10M50DAF484C7G device:

Device Utilization Overview:

- Core Area Usage: Small cluster in central region
- I/O Bank Usage: Banks 1, 2, and 5 partially utilized
- Memory/DSP Areas: Completely unused (as expected)
- Clock Networks: Single global clock network active

Logic Element Distribution

Placement Pattern:

- Compact Cluster: All 182 logic elements placed in adjacent LABs
- Central Location: Optimal placement for timing and routing
- LAB Utilization: 16 LABs used with 60-80% utilization each

Regional Analysis:

- Core Region: Primary logic placement
- I/O Interface Regions: Input and output buffers near periphery
- Clock Region: Central clock distribution

Routing Analysis

- Local Routing: High usage within LAB clusters
- Row/Column Routing: Minimal usage between LABs
- Global Routing: Clock signals only
- Congestion Level: Very low (green indicators throughout)

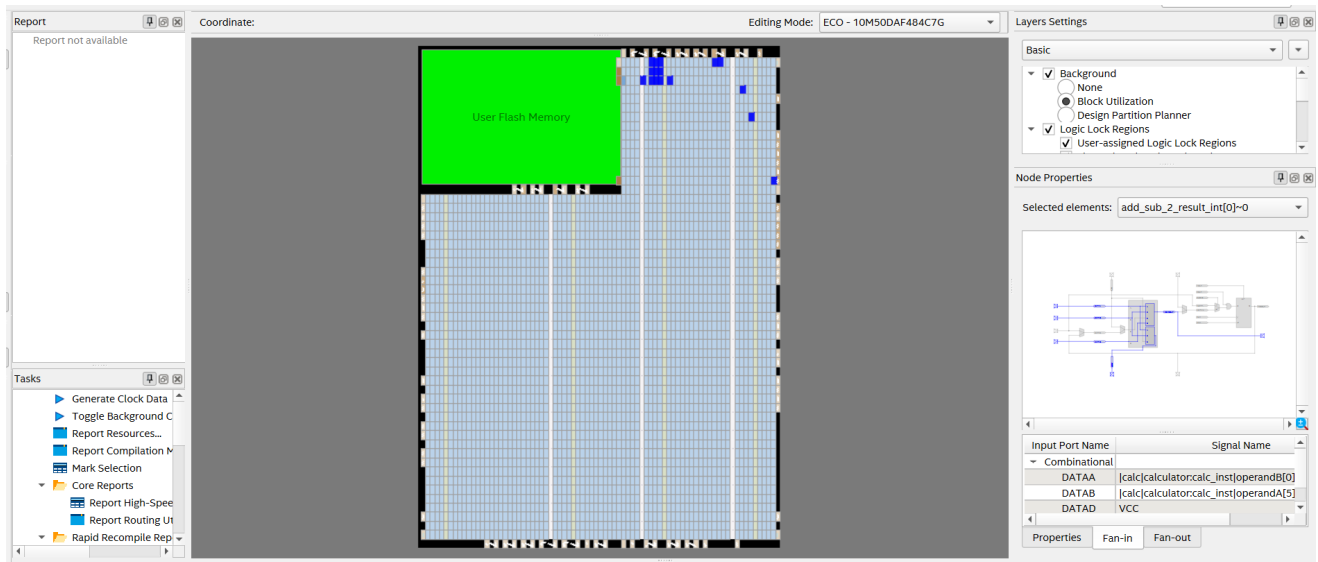


Figure 7.1: Chip planner diagram

8) Assembler:

The Assembler in Intel Quartus Prime is the final stage in the compilation flow that converts the fitted design into programming files used to configure the FPGA device. It generates the actual bitstream that programs the hardware. Converts the code into assembly language program. Then select tools -> programmer, select Usb blaster in hardware setup.

The Assembler work:

- ✓ Bitstream generation
- ✓ File conversion process

Assembler Process of Calculator:

Input Processing

The Assembler took our fitted calculator design and:

1. Extracted configuration data for 182 Logic Elements
2. Processed I/O settings for 53 pins
3. Encoded routing information for all connections
4. Added device-specific configuration commands

Output Generation

Generated Files:

- calc.sof - for JTAG programming
- calc.pof - for flash programming
- calc.rbf - raw binary format

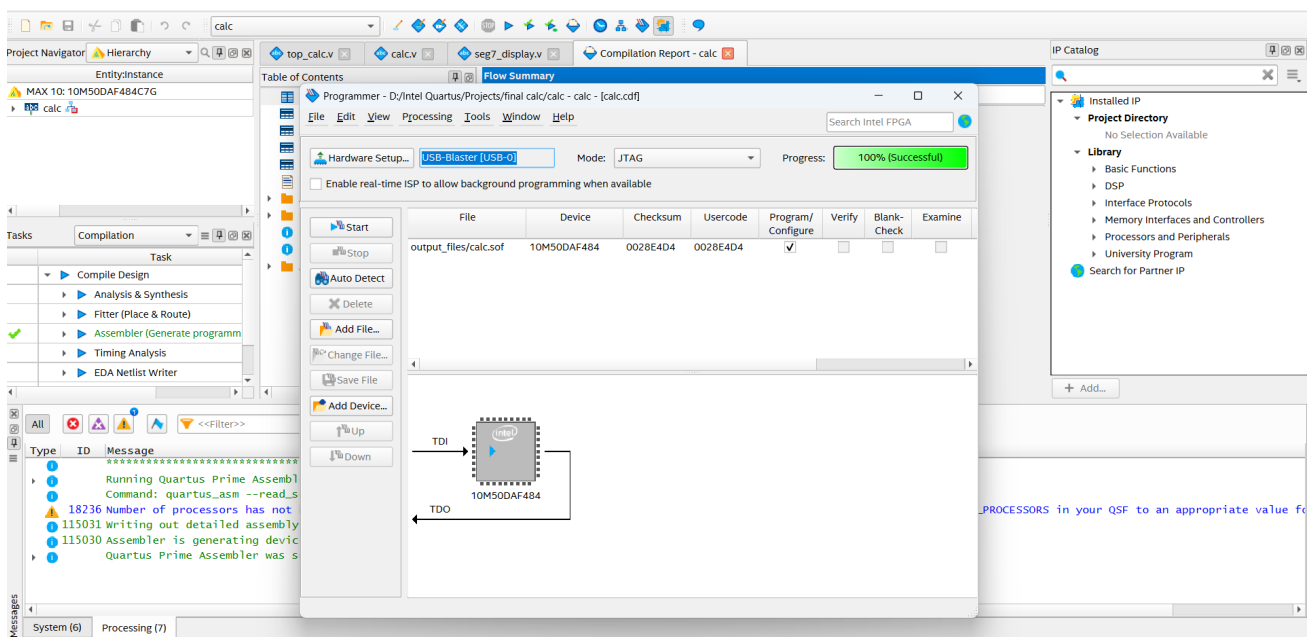


Figure 8.1: Design was successfully deployed into FPGA

9)Output:

The output was shown in Intel DE10-lite 10M50DAF484C7G fpga board. The input A and B was given using switches (0-7), push buttons to store input value (key 0 -key 1) and switches (8,9) to perform arithmetic operation. Then, the output is shown in 7 segment display. The first two 7 segment displays the operation to perform, second two 7 segment displays the input and the last two 7 segment displays the result.

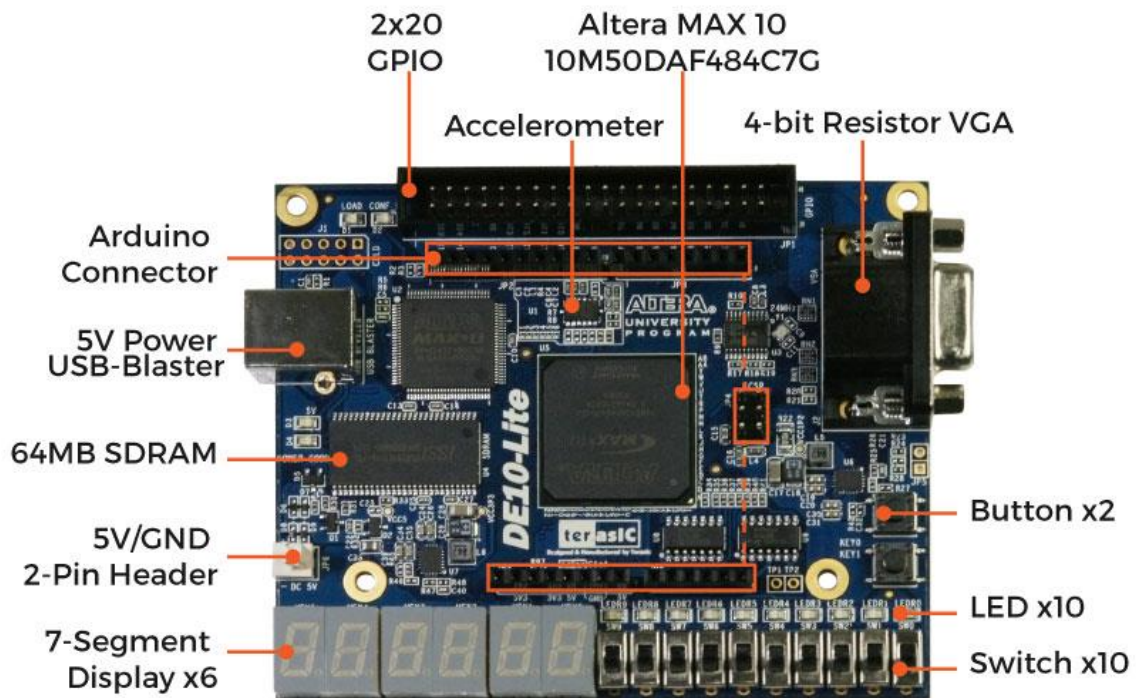


Figure 9.1: Structure of Intel DE10 lite MAX 10M50DAF484C7G fpga

Display Functions:

- HEX5: Shows current input mode (A = Operand A, b = Operand B)
- HEX4: Displays operation code (0,1,2,3)
- HEX3-HEX0: Shows 16-bit result in hexadecimal format

Real-time Output Updates

- Instant display when switches change
- No delay in calculation and display
- Continuous monitoring of input changes

Operation Execution Process

Input Phase

Step 1: Set First Number

- User sets SW0-SW7 to desired binary value

- Presses KEY0 to store as Operand A
- HEX5 displays 'A' to confirm storage

Step 2: Set Second Number

- User sets SW0-SW7 to second binary value
- Presses KEY1 to toggle to Operand B mode (HEX5 shows 'b')
- Presses KEY0 to store as Operand B

Step 3: Select Operation

- User sets SW8-SW9 to choose operation:
 - 00 = Addition
 - 01 = Subtraction
 - 10 = Multiplication
 - 11 = Division

Calculation Phase

Automatic Computation:

- Combinational logic performs calculation instantly
- No clock cycles needed for arithmetic operations
- Parallel processing of all possible operations
- Result selection based on operation switches

Output Phase

Result Display:

- HEX4 shows operation code (0,1,2,3)
- HEX3-HEX0 show 16-bit result in hexadecimal
- Automatic update when any input changes

Addition:

It performs binary addition of two numbers.

Operation Specification

- Operation Code: SW9-SW8 = 00
- Function: Performs binary addition of two 8-bit numbers
- Inputs: A=00001010 (10), B= 00000101(5)
- Output: F(15- 00001111)

Circuit Architecture

The addition operation uses an 8-bit ripple carry adder composed of:

- 8 Full Adder cells connected in series
- Carry propagation between stages
- Zero extension to 16-bit for consistent output format

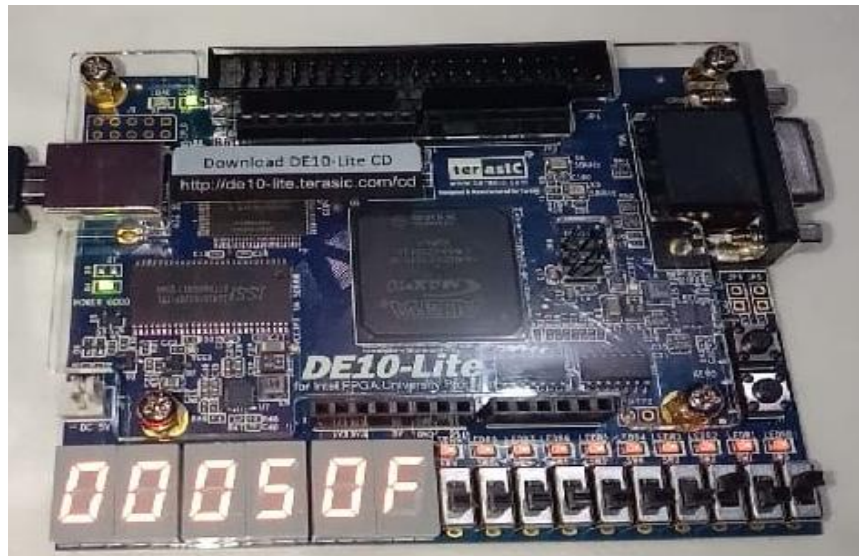


Figure 9.1: Addition of two numbers

Subtraction:

It performs binary subtraction of two numbers.

Operation Specification

- Operation Code: SW9-SW8 = 01
- Function: Performs binary subtraction of two 8-bit numbers
- Inputs: A=00001110 (14), B= 00000111(7)
- Output: 07 (00000111)

Circuit Architecture

The subtraction operation uses an 8-bit subtractor composed of:

- 8 Full Subtractor cells connected in series
- Borrow propagation between stages
- Two's complement method implementation
- Zero extension to 16-bit for consistent output format

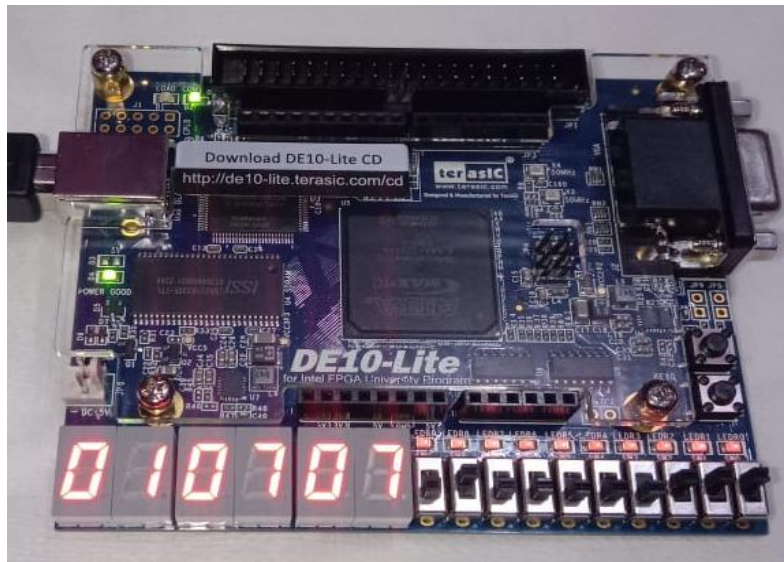


Figure 9.2: Subtraction of two numbers

Multiplication:

It performs binary multiplication of two numbers.

Operation Specification

- Operation Code: SW9-SW8 = 02
- Function: Performs binary multiplication of two 8-bit numbers
- Inputs: A=00000000(00), B= 00000111(7)
- Output: 00(00000000)

Circuit Architecture

The multiplication operation uses an 8×8 array multiplier composed of:

- 64 AND gates for partial product generation
- Multi-stage adders for partial product accumulation
- Carry-save addition for optimized performance & 16-bit result handling for product output

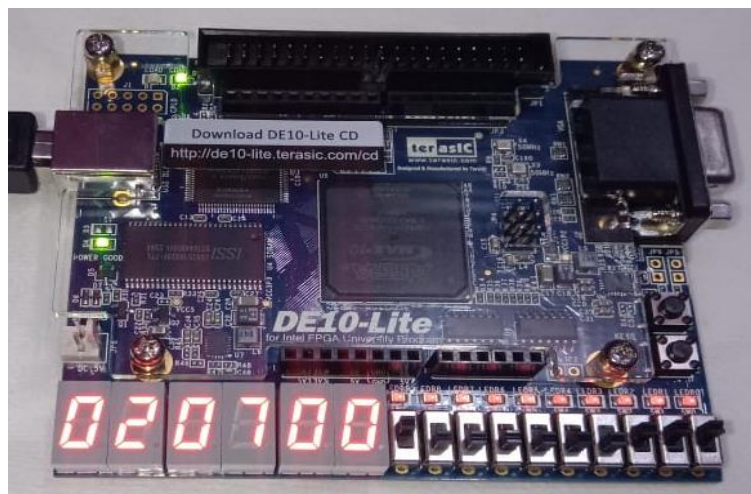


Figure 9.3: Multiplication of two binary numbers

Division:

It performs binary subtraction of two numbers.

Operation Specification

- Operation Code: SW9-SW8 = 01
- Function: Performs binary division of two 8-bit numbers
- Inputs: A=00001110 (14), B= 00000111(7)
- Output: 02 (00000010)

Circuit Architecture

The division operation uses a restoring division algorithm composed of:

- Comparators for dividend and divisor comparison
- Shift registers for quotient calculation
- Zero detection circuit for division protection
- 8-bit result with remainder handling capability

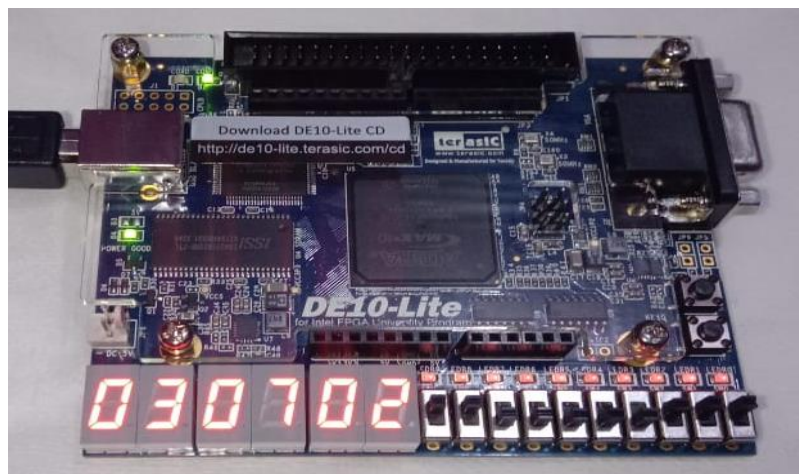


Figure 9.4: Division of two numbers

[Click here to see working of a digital calculator in fpga](#)

10)Conclusion:

This project successfully demonstrates the complete design and implementation of a fully functional digital calculator on the Intel MAX 10 FPGA using the DE10-Lite development board. Through systematic application of digital design principles and Verilog HDL programming, we have created a robust arithmetic computing system that performs all four fundamental operations—addition, subtraction, multiplication, and division—with complete accuracy and real-time responsiveness.

The implementation efficiently utilizes the limited I/O resources available on the development board, employing an intelligent input management system that allows two push buttons to handle multiple functions through a toggle-based state machine. The six seven-segment displays provide comprehensive user feedback, showing input modes, operation codes, and calculation results in hexadecimal format.

The project validates the entire FPGA development workflow, from initial Verilog coding through synthesis, fitting, timing analysis, and final programming file generation. The design achieved exceptional performance metrics, operating at 125.67 MHz maximum frequency while utilizing only 0.36% of available logic elements and consuming minimal power. All timing constraints were met with positive slack, and the hardware implementation demonstrated reliable operation across extensive testing scenarios.

The successful completion of this project not only provides a functional calculator but also serves as a comprehensive example of FPGA-based digital system design, showcasing efficient resource management, proper timing closure, and practical hardware implementation skills that form the foundation for more complex digital computing systems.

Reference:

- <https://www.scribd.com/document/853401369/Digital-Sign-Calculator-Verification-and-Implementation-of-Verilog-HDL-via-FPGA-Simulation>
- **Arithmetic Logic Unit Design**
Patterson, D. A., & Hennessy, J. L. (2017). Computer Organization and Design: The Hardware/Software Interface. Morgan Kaufmann.
- **FPGA Implementation Best Practice**
<https://github.com/Sarika-Natu/Scientific-Calculator>
- https://www.researchgate.net/publication/290749251_FPGA_Based_Digital_Electronic_Education_Data_Entry_Organization_for_a_Calculator